# FPGA-Friendly Compact and Efficient AES-like 8x8 S-Box

Ahmet Malal[a,b], Cihangir Tezcan[a]

[a]Graduate School of Informatics, Department of Cyber Security, Middle East Technical University, Ankara, 06800, Turkey
[b]ASELSAN Inc., Ankara, 06800, Turkey

ARTICLE INFO

*Keywords*:
AES
Rijndael S-Box
Compact S-Box
FPGA Implementation
Finite Field
Group Isomorphism

ABSTRACT

One of the main layers in the Advanced Encryption Standard (AES) is the substitution layer, where an $8 \times 8$ S-Box is used 16 times. The substitution layer provides confusion and makes the algorithm resistant to cryptanalysis techniques. Therefore, the security of the algorithm is also highly dependent on this layer. However, the cost of implementing $8 \times 8$ S-Box on FPGA platforms is considerably higher than other layers of the algorithm. Since S-Boxes are repeatedly used in the algorithm, the cost of the algorithm highly comes from the substitution layer. In 2005, Canright used different extension fields to represent AES S-Box to get FPGA-friendly compact designs. The best optimization proposed by Canright reduced the gate-area of the AES S-Box implementation by 20%.

In this study, we use the same optimization methods that Canright used to optimize AES S-Box on hardware platforms. Our purpose is not to optimize AES S-Box; we aim to create another $8 \times 8$ S-Box which is strong and compact enough for FPGA platforms. We create an $8 \times 8$ S-Box using the inverse field operation as in the case of AES S-Box. We use another irreducible polynomial to represent the finite field and get an FPGA-friendly compact and efficient $8 \times 8$ S-Box. The finite field we propose provides the same level of security against cryptanalysis techniques with a 3.125% less gate-area on Virtex-7 and Artix-7 FPGAs compared to Canright's results. Moreover, our proposed S-Box requires 11.76% less gate on Virtex-4 FPGAs. These gate-area improvements are beneficial for resource-constraint IoT devices and allow more copies of the S-Box for algorithm parallelism. Therefore, we claim that our proposed S-Box is more compact and efficient than AES S-Box. Cryptographers who need an $8 \times 8$ S-Box can use our proposed S-Box in their designs instead of AES S-Box with the same level of security but better efficiency.

## 1. Introduction

Cryptography, or cryptology, is the study of securing communication in the existence of third parties. The study aims to protect the transaction of data from others. In more detail, cryptography constructs algorithms and protocols to protect sensitive data from adversaries. Secure communication has always been crucial for people and states throughout history. Sometimes it changed the course of wars and caused the collapse of great states. Therefore, the birth of cryptology goes back thousands of years. Until the 1970s, many cryptographic systems were developed, most of which were pen-and-paper methods. These systems existed as works of classical cryptography. They evolved into insecure systems with the advancement of computer science. After the 1970s, modern cryptography was born. Modern cryptographic algorithms and protocols are designed to provide security against advance computers. The Data Encryption System (DES) was one of the earliest modern cryptographic algorithms which was developed by IBM in early 1970s. Later on, new cryptanalysis techniques are discovered and the security of DES is gradually weakened. Since DES has small key size, the development of brute-force attacks and the increasing power of computers made it possible to break DES encryption in a considerable amount of time. Nowadays, it is possible to capture a DES key in days on a single GPU. For instance, table-based CUDA optimizations are used to capture a DES key using exhaustive search attacks, achieving success in 215 days with an RTX3070 device [41].

NIST announced a competition to choose a block cipher algorithm to be the successor of the Data Encryption Standard (DES) on January 2, 1997. NIST decided the specification of the algorithm. The block size of the algorithm was chosen to be 128-bit. The algorithm was supposed to have three modes that have different key sizes. The key sizes are chosen as 128, 196, and 256 bits. The competition initially began with fifteen candidates submitted from several countries. Vulnerabilities were detected in some candidates, and some were eliminated due to poor performance. NIST eliminated ten of fifteen candidates and announced five finalist algorithms. Rijndael, Serpent, Twofish, RC6, and MARS were announced as the finalists of the competition. After three rounds of elimination stage, the Rijndael algorithm was selected as the winner in April 2000, called later the Advanced Encryption Standard. As a result, DES was gradually replaced by more secure encryption algorithm AES (Advanced Encryption Standard), which uses larger key sizes and more complex encryption processes to provide better security.

The Rijndael algorithm was designed by Joan Daemon and Vincent Rijmen in 1997 [13]. The Rijndael algorithm is chosen thanks to its strong performance on different platforms and security resistance against cryptographic attacks. Since many cryptographers and researchers worked and analyzed the algorithm in the competition process, several papers and articles have been published about the Rijndael

ahmet.malal@metu.edu.tr (A. Malal); cihangir@metu.edu.tr (C. Tezcan)
ORCID(s): 0009-0009-4019-8617 (A. Malal); 0000-0002-9041-1932 (C. Tezcan)

This article is produced from A. Malal's Master's thesis, which served as the primary source of research and analysis for the content presented.

algorithm. So many designs and implementation strategies have been proposed with different metrics until today.

The performance of the AES algorithm is critical in many real-world applications where data security is important. High-performance implementations of AES are needed in order to provide fast and efficient encryption and decryption of sensitive data. This is particularly important in applications where large amounts of data need to be processed in real-time, such as in network security, cloud computing, and mobile devices. In addition, the performance of AES can impact the user experience of many applications. Slow encryption and decryption can lead to delays. Therefore, optimizing the performance of AES has been an important topic in cryptography, and many techniques have been developed to improve the speed and efficiency of AES implementations on both software and hardware platforms.

In recent years, processor manufacturers like Intel and AMD have started to include instruction sets for AES in their products. For example, several Intel processors have started to support AES instructions set since 2010 [15]. These instruction sets are specifically designed to accelerate AES encryption and decryption, resulting in improved performance and efficiency. By utilizing these instruction sets, software developers can take advantage of the processing power of modern CPUs, leading to faster and more secure applications. One of the recent studies reports achieving a throughput of 134.7 Gbps with an AES implementation that utilizes AES instruction sets on *Intel i7-10700F* [40]. High performance implementation of AES on CUDA devices is essential for many applications that require fast and efficient encryption and decryption of large amounts of data. CUDA-enabled devices, such as graphics processing units (GPUs), can provide significant performance improvements over traditional central processing units (CPUs) thanks to their highly parallel architecture. This makes them an ideal platform for implementing AES algorithms that can process large volumes of data in real-time.

FPGAs offer several advantages over traditional software implementations of AES, including faster processing speeds and lower power consumption. These benefits make FPGAs an attractive choice for applications that need fast encryption and decryption. Therefore, the hardware performance of AES algorithm is highly important. When designing and assessing the performance of AES on FPGA platforms, several performance metrics are taken into account. These metrics are gate-area, throughput, security, data-path width, power consumption, and more. For instance, in the study conducted by [38], a comparison is made between two different data-path widths with a focus on security against fault attacks. However, gate-area is commonly regarded as the most crucial metric, as it directly influences the cost and physical dimensions of the FPGA design. In other words, a smaller gate-area implies a more compact design, which translates to lower costs, reduced power consumption, and increased portability. Therefore, optimizing the gate-area of an AES implementation on an FPGA is a crucial task to achieve high-performance, cost-effective, and energy-efficient designs [10]. Several optimization techniques, such as hardware pipelining, parallel processing, and optimized data paths, have been developed to reduce the gate-area of AES implementations on FPGA platforms while maintaining high throughput [9]. The survey article [16] compiled a comprehensive collection of AES implementations on FPGA, encompassing a diverse range of results. Additionally, the article includes a comparative analysis in the form of a table with different metrics.

AES algorithm consists of substitution, permutation, mix-column, and add round key layers [13]. Numerous studies have been conducted to propose various modifications targeting different layers of the AES algorithm, with the aim of achieving better performance or security. For instance, [22] introduced a modified mix-column approach that resulted in a more compact design, while [31] proposed a modified permutation layer that use a small-scale confusion operation to enhance security. However, in this research, our primary focus is on the substitution layer due to its significant impact on performance and security compared to other layers.

The substitution layer, which uses S-Box, is the only non-linear layer of the algorithm. The S-Box is a critical component of AES, and its strength directly impacts the overall security of the algorithm. Therefore, the selection of an appropriate S-Box is crucial for the success of the algorithm [20]. In contrast to the conventional affine transform based S-Box of AES, alternative designs exist that employ different structures for security. In this context, [32] and [33] introduce a novel S-box design approaches that utilize coset graphs and a newly defined operation on a square matrix. In addition to security considerations, the efficient utilization of resources is also a crucial aspect in S-box design. Substitution layer is also most costly layer of the algorithm in terms of gate-area. Therefore, the gate-area of the S-Box layer is a key performance metric for FPGA implementations of AES, as it directly impacts the overall performance of the algorithm on the platform. Therefore, researchers have developed various techniques to optimize the gate-area of the S-Box layer, such as using lookup tables, pre-computation of values, and arithmetic calculation. These techniques have resulted in highly optimized S-Box implementations for FPGA platforms that can achieve high performance while minimizing the gate-area required for implementation. S-Box is implemented in various FPGAs with different techniques until today. A mux-based implementation was proposed in [28], while [18] utilized pre-calculated tables to store Galois multiplications for a compact design. A key-dependent dynamic S-box for the substitution layer of AES as an alternative to the static S-box is proposed by [14], introducing VLSI optimizations. Their proposal aims to enhance the security of the algorithm. Furthermore, S-Box is also studied for Virtex-5 and Virtex-6 FPGAs [9], [25], [28], [37]. Additionally, Canright used different extension fields to represent AES S-Box to get FPGA-friendly compact designs. The best optimization

proposed by Canright reduced the gate-area of the AES S-Box implementation by 20% [10].

The main focus of this study was on the compact implementation of the substitution layer of AES algorithm. A compact S-Box implementation is critical for FPGA platforms because it directly impacts the number of resources required for the implementation, and thus the size and cost of the FPGA device. A compact implementation of AES S-Box on FPGA can lead to higher performance, lower power consumption, and reduced cost, which are all important factors for practical applications, especially IoT and resource limited devices. Additionally, a small gate-area based implementation can also free up FPGA resources for other tasks, allowing for more complex systems to be designed within the same FPGA device or pipelining. Therefore, the importance of a compact S-Box implementation on FPGA platforms cannot be overstated. Since the S-Box of AES is strong and optimized by many researchers for different platforms, its popularity is high. Many cryptographic algorithms used AES S-Box in the following years. ARIA, Grand Cru, and Panama are examples of algorithms that use AES S-Box.

Although there have been numerous studies on optimizing AES S-Box for hardware platforms, we propose another irreducible polynomial to represent the finite field of the S-Box, which is as strong as AES S-Box but requires fewer gates on FPGAs. We aimed to propose new S-Box which can be implemented in a more compact way than AES S-Box on FPGA platforms. While optimizing the proposed S-Box, we used the same techniques that Canright used to optimize AES S-Box. Our proposed finite field representation offers the same level of security against cryptanalysis techniques, while using 3.125% less gate-area on Virtex-7 and Artix-7 FPGAs compared to Canright's results. Furthermore, on Virtex-4 FPGAs, our proposed S-Box requires 11.76% less gate-area. Our proposed S-Box can be used by cryptographers who need an 8×8 S-Box in their designs instead of the AES S-Box, offering the same level of security with better efficiency.

## 2. Advanced Encryption Standard

### 2.1. Mathematical Preliminaries

AES uses a byte as a basic unit for processing. **A byte** consists of eight bits. The AES algorithm's input and output are 128-bit, equal to 16 bytes. The bytes are represented as the concatenation of bit values from the most significant to the least significant bits. Let $a$ be a byte, then $a$ is represented as $\{a_7, a_6, a_5, a_4, a_3, a_2, a_1, a_0\}$. With a polynomial representation, each byte is represented as finite field elements.

$$\sum_{i=0}^{7} a_i x^i = a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

The hexadecimal notation is also widely used for representing bytes. **A nibble** consists of four bits. Two nibbles can be used to represent a byte. ***0x*** is used before hexadecimal notation of bytes. For example, 10100011 identifies the element $x^7 + x^5 + x + 1$. The first nibble is "***a***" and the last nibble is "***3***". 10100011 is represented as *0xa3* in hexadecimal notation.

The AES performs operations on 16 bytes in each round. The 16 bytes are ordered in a two-dimensional array of bytes, which is called **the state**. The state consists of four columns and four rows, each has four bytes.

The columns and rows of state is represented as follows ($c_i$'s denotes columns, $r_i$'s denotes rows of the states.):

$$\begin{aligned} c_0 &= s_0 s_1 s_2 s_3 & r_0 &= s_0 s_4 s_8 s_{12} \\ c_1 &= s_4 s_5 s_6 s_7 & r_1 &= s_1 s_5 s_9 s_{13} \\ c_2 &= s_8 s_9 s_{10} s_{11} & r_2 &= s_2 s_6 s_{10} s_{14} \\ c_3 &= s_{12} s_{13} s_{14} s_{15} & r_3 &= s_3 s_7 s_{11} s_{15} \end{aligned}$$

- **Galois Field**

**Definition 2.1 (Galois Field).** *A Galois field GF is a field with finite number of elements. The number of elements in the finite field is called cardinality. A Galois Field with cardinality q is denoted as GF(q) or* $\mathbb{F}_q$.

The elements of Galois field *GF(q)* is defined as

$$GF(q) = (0, 1, 2, ..., q-2, q-1)$$

**Example**: List the elements of $GF(2^4)$ in polynomial notation.

$$\begin{aligned} GF(2^4) = &\{0, 1\} \quad \cup \\ &\{x, x+1\} \quad \cup \\ &\{x^2, x^2+1, x^2+x, x^2+x+1\} \quad \cup \\ &\{x^3, x^3+1, x^3+x, x^3+x+1, x^3+x^2, \\ &x^3+x^2+1, x^3+x^2+x, x^3+x^2+x+1\} \end{aligned}$$

There are $2^4 = 16$ elements, where each of the element is polynomial of degree at most 3 and coefficients are in $\mathbb{Z}_2$.

- XOR operation

The operation XOR is denoted with $\bigoplus$. It is simply bit-wise addition in modulo 2. The XOR of two bytes is performed with addition of each bit correspondingly in modulo 2. For example, $0x2a \oplus 0x32 = 0x18$.

- Multiplication

Multiplication of two elements is performed in Galois Field ($2^8$), and it is denoted as •. After a standard polynomial multiplication, an irreducible polynomial is applied to the result as the modulo operation. The designers of the AES chose the irreducible polynomial to be $x^8 + x^4 + x^3 + x + 1$, which is the smallest irreducible polynomial degree of 8.

For example, $0x82 \bullet 0x21$:

$$\begin{aligned}(x^7+x)\bullet(x^5+1) &= x^{12}+x^7+x^6+x\\ &= x^4x^8+x^7+x^6+x\\ &= x^4(x^4+x^3+x+1)+x^7+x^6+x\\ &= x^8+x^7+x^5+x^4+x^7+x^6+x\\ &= x^4+x^3+x+1+x^5+x^4+x^6+x\\ &= x^6+x^5+x^3+1\end{aligned}$$

$x^6+x^5+x^3+1$ is equal to 01101001 in binary system, which is *0x69*.

**Definition 2.2 (Irreducible Polynomials).** *A polynomial $f$ over $F[x]$ is irreducible if the only factorization of it are the trivial ones (one and the polynomial itself).*

**Definition 2.3 (Primitive Element).** *A group $G$ is called cyclic if there is an element $e \in G$ such that for any $x \in G$ there is some integer $i$ with $x = e^i$. The element $e$ is called primitive element (or generator), and it is written as $G =< e >$.*

**Definition 2.4 (Primitive Polynomials).** *A polynomial which generates all elements of an extension field is called a primitive polynomial.*

**Example:** Show that $x^4+x+1$ is a primitive polynomial over $GF(2)$.

We need to prove that there is an element $e$ such that $e$ can generate all the elements of $GF(2^4)$. We have shown the elements of $GF(2^4)$ above.
Let generator be $x$.

$$\begin{aligned} x^1 &= x & x^9 &= x^3+x\\ x^2 &= x^2 & x^{10} &= x^2+x+1\\ x^3 &= x^3 & x^{11} &= x^3+x^2+x\\ x^4 &= x+1 & x^{12} &= x^3+x^2+x+1\\ x^5 &= x^2+x & x^{13} &= x^3+x^2+1\\ x^6 &= x^3+x^2 & x^{14} &= x^3+1\\ x^7 &= x^3+x+1 & x^{15} &= 1\\ x^8 &= x^2+1 \end{aligned}$$

We replaced $x^4$ with $x+1$ since they are equal in the given field. As we can see that all the elements are generated. To sum up, $x$ is the primitive element and $x^4+x+1$ is a primitive polynomial over $GF(2)$.

**Definition 2.5 (Irreducible Primitive Polynomials).** *If a polynomial $f$ is both irreducible and primitive, then it is called irreducible primitive polynomial.*

For AES S-Box, the field $GF(2^8)$ is used. Now we know that the field has exactly 256 elements and each element of the field can be represented by a polynomial degree at most 7. The coefficient of polynomials are in $\mathbb{Z}_2$.

- **Field Isomorphism**

The optimization of [10] is mainly constructed on isomorphism of finite fields. Thanks to irreducible polynomial of AES S-Box, Canright proposed different way to construct AES S-Box by using field isomorphism.

**Definition 2.6 (Isomorphism).** *Assume there are two fields $\left(\mathbb{F}_1,(+,\times)\right)$ and $\left(\mathbb{F}_2,(\oplus,\cdot)\right)$ with the same number of elements. A one-to-one map $f : \mathbb{F}_1 \mapsto \mathbb{F}_2$ is called an isomorphism from $\left(\mathbb{F}_1,(+,\times)\right)$ onto $\left(\mathbb{F}_2,(\oplus,\cdot)\right)$, if for any $a,b \in \mathbb{F}_1$*

$$\begin{aligned} f(a+b) &= f(a)\oplus f(b),\\ f(a\times b) &= f(b)\cdot f(b)\end{aligned}$$

*Then $\left(\mathbb{F}_1,(+,\times)\right)$ and $\left(\mathbb{F}_2,(\oplus,\cdot)\right)$ are called isomorphic fields to each other.*

**Example:** Consider the field $\left(\mathbb{Q}(\sqrt{2}),(+,\cdot)\right)$ whose elements are in the form $a+b\sqrt{2}$, and the field $\left(\mathbb{Q}(\sqrt{5}),(+,\cdot)\right)$ whose elements are in the form $c+d\sqrt{5}$, where $a,b,c,d \in \mathbb{Q}$.

The function $f : \mathbb{Q}(\sqrt{2}) \to \mathbb{Q}(\sqrt{5})$ defined by:

$$f(a+b\sqrt{2}) = a+b\sqrt{10}$$

is claimed to be a field isomorphism. In order to be a field isomorphism, the following three properties must be satisfied by the function $f$:

$f$ **is a one-to-one:** Since $f$ maps every element in $\mathbb{Q}(\sqrt{2})$ to a distinct element in $\mathbb{Q}(\sqrt{5})$, and vice versa, $f$ is a one-to-one.

$f$ **satisfies addition:** For any $x,y \in \mathbb{Q}(\sqrt{2})$ assume $x = a+b\sqrt{2}$ and $y = c+d\sqrt{2}$, the following equation is verified:

$$\begin{aligned} f(x+y) &= f((a+b\sqrt{2})+(c+d\sqrt{2}))\\ &= f((a+c)+(b+d)\sqrt{2})\\ &= (a+c)+(b+d)\sqrt{10}\\ &= (a+b\sqrt{10})+(c+d\sqrt{10})\\ &= f(x)+f(y)\end{aligned}$$

Thus, $f$ satisfies addition.

$f$ **satisfies multiplication:** For any $x,y \in \mathbb{Q}(\sqrt{2})$ assume $x = a+b\sqrt{2}$ and $y = c+d\sqrt{2}$, the following equation is verified:

$$\begin{aligned} f(xy) &= f((a+b\sqrt{2})(c+d\sqrt{2}))\\ &= f((ac+2bd)+(ad+bc)\sqrt{2})\\ &= (ac+2bd)+(ad+bc)\sqrt{10}\\ &= (a+b\sqrt{10})(c+d\sqrt{10})\end{aligned}$$

$$= f(x)f(y)$$

Thus, $f$ satisfies multiplication. Since $f$ is a one-to-one that preserves addition and multiplication, it is a field isomorphism.

## 2.2. Algorithm Specification

The AES algorithm takes 128-bit as input, performs a series of operations, and generates 128-bit ciphertext. The round function of AES consists of four different layers; each has a different property. According to key size, AES has a different number of rounds. Each round is similar to the other, except for the last round. There is no Mix-Columns operation in the last round. AES have 128, 196, and 256-bit key size versions and each has 10, 12, and 14 rounds respectively. The block size of the algorithm is fixed for all version, which is 128-bit.

### *2.2.1. Encryption / Decryption*

- **Sub-Bytes / Inverse Sub-Bytes**

Each state byte is substituted with another byte by using the substitution table, which is called S-Box. Transformation of bytes provides non-linearity, making the algorithm strong against differential and linear attacks. The S-Box of the algorithm is generated with an affine transform.

Inverse Sub-Bytes is the inverse function of Sub-Bytes. The inverse multiplication is performed on the result of the affine transformation for each byte of the state.

- **Shift-Rows / Inverse Shift-Rows**

The state rows are cyclically shifted to the left with offset bytes. The first row remains the same. The second row's bytes are cyclically shifted to the left. The third row's bytes are shifted two bytes. Bytes in the last row are cyclically moved three bytes to the left. This layer provides byte permutation, which provides diffusion. The inverse of the Shift-Rows is the same as the Shift-Rows, except for shifting direction. The cyclic right shift is applied on the rows in the inverse Shift-Rows.

- **Mix-Columns / Inverse Mix-Columns**

Constant matrix multiplication is performed on the state. Each column is separately multiplied by the rows of the constant matrix. The inverse of the Mix-Columns is also a constant matrix multiplication. For Inverse Mix-Columns, the inverse of the matrix is used.

- Mix-Columns

$$\begin{bmatrix} s'_0 & s'_4 & s'_8 & s'_{12} \\ s'_1 & s'_5 & s'_9 & s'_{13} \\ s'_2 & s'_6 & s'_{10} & s'_{14} \\ s'_3 & s'_7 & s'_{11} & s'_{15} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \bullet \begin{bmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{bmatrix}$$

- Inverse Mix-Columns

$$\begin{bmatrix} s'_0 & s'_4 & s'_8 & s'_{12} \\ s'_1 & s'_5 & s'_9 & s'_{13} \\ s'_2 & s'_6 & s'_{10} & s'_{14} \\ s'_3 & s'_7 & s'_{11} & s'_{15} \end{bmatrix} = \begin{bmatrix} e & b & d & 9 \\ 9 & e & b & d \\ d & 9 & e & b \\ b & d & 9 & e \end{bmatrix} \bullet \begin{bmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{bmatrix}$$

- **AddRoundKey / Inverse AddRoundKey**

The state is XORed with the 128-bit round key in this layer. The AddRoundKey and Inverse AddRounKey are the same since the inverse of the XOR operation is XOR itself.

### *2.2.2. Key Generation*

The Key Expansion function is a crucial part of the AES algorithm that generates a series of round keys based on the initial key. The number of rounds in the algorithm depends on the size of the initial key. For a 128-bit key, the algorithm runs for 10 rounds. During each round, a different round key is used to transform the data. The Key Expansion function generates these round keys by performing a series of operations on the initial key.

**Rcon** is an array of round constant bytes used in the key schedule of AES, which is defined as:

$$Rcon = \{0x01, 0x02, 0x04, 0x08, 0x10 \\ 0x20, 0x40, 0x80, 0x0x1B, 0x36\}$$

- **Sub-Word**

The Sub-Word function is the sub-version of Sub-Bytes. The function takes four bytes, calculates the S-Box function of each byte, and returns them.

- **Rot-Word**

The Rot-Word function is similar with Shift-Rows. It takes four bytes and performs cyclic shift to left.

## 2.3. Performance Evaluation of AES on Different Platforms

After the Rijndael Algorithm was chosen as a standard, it is used in various systems that require security. As the AES algorithm has become increasingly popular, the number of researchers trying to optimize it has increased. Numerous implementation techniques have been published by considering various metrics. Throughput, power consumption, and gate-area are only three of the metrics. We summarized recent software and hardware performance analysis results on the AES algorithm in this subsection. Table 1 shows the recent AES implementation result on CUDA devices. The table was sorted according to Gbps per Watt. Many researchers also proposed optimized implementation of AES for Intel CPUs. The best result is 2.072 Gbps/Watt, which is obtained by [40]. The designs which focused on the throughput per slice on FPGA are listed in Table 2. These implementations are suitable for resource constrained devices. Table 3 shows the fastest AES implementations on FPGA. The table is sorted with respect to gigabit per second.

**Table 1**
Summary of AES-128 encryption performance on different CUDA devices

| Device | Architecture | Gbps/W | Gbps | Design |
|---|---|---|---|---|
| Nvidia RTX 2070 Super | Turing | 0.236 | 50.8 | [4] |
| Nvidia GTX 1070 | Pascal | 1.427 | 214.0 | [3] |
| Nvidia GTX 1080 | Pascal | 1.555 | 279.9 | [1] |
| Nvidia RTX 2070 | Turing | 1.771 | 310.0 | [3] |
| Nvidia GTX 970 | Maxwell | 2.174 | 315.2 | [40] |
| Nvidia Tesla P100 | Pascal | 2.423 | 605.9 | [26] |
| Nvidia RTX 2070 Super | Turing | 4.087 | 878.6 | [40] |

**Table 2**
Throughput comparison of AES per slice (LUT) on different FPGA devices

| Device | BRAM | Slice (LUT) | Throughput (GBit/s) | TPS(Mbps/Slice) | Design |
|---|---|---|---|---|---|
| Virtex-6 | 16 | 4926 | 1.815 | 0.368 | [12] |
| Virtex-7 | 0 | 2444 | 5.306 | 2.17 | [17] |
| Virtex-5 | 0 | 1223 | 3.7 | 2.76 | [7] |
| Virtex-5 | 0 | 950 | 4.1 | 4.315 | [9] |
| Virtex-5 | 0 | 798 | 4.34 | 5.43 | [30] |
| Virtex-5 | 2 | 459 | 4.262 | 9.29 | [21] |

# 3. Compact AES S-Box Implementation for Hardware Platforms

## 3.1. Construction of S-Box

**Definition 3.1 (Circulant Matrix).** *An $n \times n$ matrix called circulant if it is in the form:*

$$M = \begin{pmatrix} c_{n-1} & c_0 & c_1 & . & . & c_{n-4} & c_{n-3} & c_{n-2} \\ c_{n-2} & c_{n-1} & c_0 & . & . & . & c_{n-4} & c_{n-3} \\ . & c_{n-2} & c_{n-1} & . & . & . & . & c_{n-4} \\ . & . & c_{n-2} & . & . & . & . & . \\ . & . & . & . & . & . & c_1 & . \\ c_2 & . & . & . & . & . & c_0 & c_1 \\ c_1 & c_2 & . & . & . & . & c_{n-1} & c_0 \\ c_0 & c_1 & c_2 & . & . & . & c_{n-2} & c_{n-1} \end{pmatrix}$$

The transpose of this form also constructs *circulant* matrix. The cyclic permutation of the first row creates other rows of the matrix. The circulant matrix that contains only 0s and 1s is called *circulant binary matrix*.

The polynomial representation of the matrix is as follow:

$$f(x) = c_{n-1}x^{n-1} + c_{n-2}x^{n-2} + ... + c_1x + c_0$$

The Rijndael S-Box use the following circulant binary matrix.

$$M = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

The polynomial representation of the Rijndael S-Box is

$$f(x) = x^7 + x^3 + x^2 + x + 1$$

The polynomial is also denoted in hexadecimal form $0x8f$. In this study, the hexadecimal notation of circulant matrices is used for simplicity.

### *3.1.1. Affine Transform*

The S-Box function of AES is generated with an affine transform. The affine transformation formula is given below.

$$S(x) = M \times x^{-1} \oplus c,$$

where $M$ is the $8 \times 8$ circulant binary matrix and $c$ is a constant byte.

***Step 1:*** Find the multiplicative inverse of $a$ in $GF(2^8)$

$$b = a^{-1}, \quad (1)$$

**Table 3**
Throughput comparison of AES per seconds on FPGA devices

| Device | BRAM | Slice (LUT) | Throughput (GBit/s) | TPS (Mbps/Slice) | Design |
|---|---|---|---|---|---|
| Virtex-5 | 0 | 8896 | 25.89 | 2.91 | [34] |
| Virtex-5 | 20 | 4491 | 42.62 | 9.49 | [21] |
| Virtex-6 | 0 | 18854 | 44.074 | 3.71 | [43] |
| Virtex-2 Pro | 80 | 11398 | 57.28 | 5.026 | [19] |

(if a = 0, then b = 0).

***Step 2:*** Calculate the following affine transformation.

$$\begin{pmatrix} s_7 \\ s_6 \\ s_5 \\ s_4 \\ s_3 \\ s_2 \\ s_1 \\ s_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} b_7 \\ b_6 \\ b_5 \\ b_4 \\ b_3 \\ b_2 \\ b_1 \\ b_0 \end{pmatrix} \oplus \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

$s_i$'s and $b_i$'s are the bits of $s$ and $b$ correspondingly and $0 \leq i < 7$.

## 3.2. Implementation Methods of S-Boxes for FPGA Platforms

S-Box implementation on FPGA platforms is a critical factor in the performance and efficiency of block ciphers. There are several methods to implement S-Boxes on FPGAs, including RAM-based, lookup tables (LUTs), arithmetic-based methods,

### *3.2.1. RAM-Based Implementation*

There are 256 different input values for an S-Box function. The output of the S-Box function is a byte. Therefore, a 256-byte table is enough to keep an S-Box. Block Rams on FPGA platforms can be used to store the S-Boxes. RAMs are the memory spaces in FPGA, which can be written with data or retrieved as data. The type of memory which is not writable is called ROM (read-only memory). We do not need to write anything to memory for S-Box tables; we just read the data. For convention, we use RAM instead of ROM.

Several designers prefer to use RAM-based implementation [5], [21]. Although keeping S-Boxes in a Block RAM increases the throughput performance of the algorithm [35], using RAMs for S-Boxes also has many drawbacks.

Depending on FPGA, a RAM can store a block of 18-Kbit or 36-Kbit data. For every 8-bit of data, one parity bit is used. Therefore, 18-Kbit RAMs can store 16-Kbit, and 36-Kbit RAMs can store 32-Kbit. Since only a single data can be retrieved from RAM in one cycle, one RAM can be used for only one byte [2]. AES state has 16 bytes. Therefore 16 Block RAMs are needed to implement the S-Box layer in one cycle. Only 256 bytes of the RAM is used to keep an S-Box; the rest of the RAM will be useless. The number of RAM is limited for each FPGA, and RAMs have different purpose of usage rather than storing small tables. Therefore, using RAMs for S-Box implementation is not common.

- Singe Port Block-RAM

The Singe Port Block-RAM has only one interface. These types of RAMs can read or write only a single data in each cycle. This is the simplest RAM configuration for FPGA platforms to retrieve data. According to *write enable (we)* signal, a data is retrieved or written. If the *write enable* signal is $high(1)$, *data input (dina)* is written into *address (addra)* index of RAM. If the *write enable* signal is $low(0)$, the data in the *data input (dina)* index is retrieved from RAM with *data out (dout)* signal. *Enable (en)* signal enables and disables the RAM.

### *3.2.2. Logic-Based Implementation*

Instead of storing S-Box in RAMs, the Look-up Tables (LUT) can be also used for this purpose. There are different types of LUTs; in this study, we focused on 4-LUT and 6-LUT since the FPGAs that we worked on use 4-LUT and 6-LUT types of LUTs. According to each input combination, one-bit output is kept in the truth table. If we write the S-Box as a byte array in memory, the synthesizer converts this S-Box into LUTs. According to our studies, the FPGAs that use 4-LUTs as a primitive need 64 Look-up tables to represent 256-byte S-Box table. For the FPGAs whose primitives are 6-LUTs need 40 Look-up tables to implement the S-Box. The Naive Look-up table implementation refers to this implementation style.

- 4-LUT

In 4-LUT, there are four different bits as an input. According to each combination of inputs, the output bit is stored in the Look-up table. In this case, the size of the table is $2^4 = 16$. For more information see [2].

- 6-LUT

In 6-LUT, there are six different bits as an input. According to each combination of inputs, the output bit is stored in the Look-up table. In this case, the size of the table is $2^6 = 64$. It consists of two 5-LUTs and one multiplexer. For more information see [2].

### *3.2.3. Arithmetic-Based Implementation*

The last approach that we mentioned in this study is arithmetic-based implementation. The S-Box function is performed directly by implementing the multiplicative inverse and affine transform. The affine transform is straightforward and costs less gate-area on hardware platforms. However, taking the multiplicative inverse in $GF(2^8)$ is expensive. Several designs use arithmetic-based implementation. Optimizing the operations on the Galois field reduces the cost of implementing S-Box. [36] used field isomorphism properties and proposed a new method to calculate the multiplicative inverse in $GF(((2^2)^2)^2)$ instead of $GF(2^8)$. Later, [10] optimized [36] results and proposed new isomorphism matrices to represent S-Box.

In this study, we used a composite field approach to find a new S-Box that costs less gate-area than Canright results. Our purpose was not to optimize AES S-Box in this study. We aimed to find another S-Box that could be implemented with fewer gates on the hardware platform. The security that S-Box provides against cryptanalysis techniques is also concerned. In this work, we propose a new S-Box that can be implemented with fewer gates and provides at least the same security level of AES's S-Box.

# 4. The Proposed Compact S-Box for Hardware Platforms

AES can be implemented in a variety of ways on hardware platforms. The implementation details are highly dependent on a target platform and specific FPGA. Pipelining, unrolling, datapath width, and frequency requirements are the methods/aspects that can affect the architecture of the implementation.

The approach of implementing S-Box is the most critical part of these design methods [9]. This is because the S-Box layer is the most expensive to implement among the other levels of AES on hardware. There are three approaches to implement an S-Box on hardware platforms: logic-based, arithmetic-based, and RAM-based implementations. The details of these approaches are explained in the previous section 3.2.

The S-Box of AES is already implemented in a very compact way on hardware by using composite field approach of Galois field $GF(2^8)$ [10], which is one of the approaches of arithmetic-based implementation. In this study, we aim to look for different primitive polynomial of finite field $GF(2^8)$, which provides an S-boz that has at least the same security level of AES's S-Box against linear and differential cryptanalysis and costs less gate-area on hardware platforms.

## 4.1. Implementation Details

Irreducible polynomials are the polynomials which cannot be expressed by the multiple of two non-constant polynomials. Primitive polynomials have at least one generator, as we defined in the previous section in detail. A polynomial which is both irreducible and primitive is called primitive irreducible polynomial.

By Gauss's formula, there are 30 different primitive irreducible polynomials of degree 8 over $GF(2^8)$ [39]. The list of the primitive irreducible polynomials is given in the Table 4. The polynomial of AES S-Box is the first polynomial of the table, which is the smallest $8^{th}$ degree primitive.

$$S(x) = M * x^{-1} \oplus c,$$

is the S-Box affine transformation formula, where $M$ is the $8 \times 8$ circulant binary matrix and $c$ is a constant byte.

Using composite field approach, Canright optimized the taking inverse of $x$ over $GF(2^8)$, $M$ and $c$ are not considered in the optimization part. There are 256 different $8 \times 8$ circulant binary matrices and 256 different constant values. As we explained above, there are 30 different irreducible primitive polynomials. Therefore, $30 \times 256 \times 256$ different look-up tables can be created in the considered affine transformation form. The look-up tables, which do not have one-to-one property, are ignored in this study.

- **Composite Field Approach**

The notation of [10] is used for compatibility. Let $g$ be an element of $GF(2^8)$ and represented as $g = g_7 g_6 g_5 g_4 g_3 g_2 g_1 g_0$, where $g_i \in GF(2)$. $g$ is also represented as

**Table 4**
The list of all $8^{th}$ degree primitive irreducible polynomials in $GF(2^8)$

| | Hexadecimal Form | Polynomial |
|---|---|---|
| 1 | $0x11b$ | $x^8 + x^4 + x^3 + x^1 + 1$ |
| 2 | $0x11d$ | $x^8 + x^4 + x^3 + x^2 + 1$ |
| 3 | $0x12b$ | $x^8 + x^5 + x^3 + x^1 + 1$ |
| 4 | $0x12d$ | $x^8 + x^5 + x^3 + x^2 + 1$ |
| 5 | $0x139$ | $x^8 + x^5 + x^4 + x^3 + 1$ |
| 6 | $0x13f$ | $x^8 + x^5 + x^4 + x^3 + x^2 + x^1 + 1$ |
| 7 | $0x14d$ | $x^8 + x^6 + x^3 + x^2 + 1$ |
| 8 | $0x15f$ | $x^8 + x^6 + x^4 + x^3 + x^2 + x^1 + 1$ |
| 9 | $0x163$ | $x^8 + x^6 + x^5 + x^1 + 1$ |
| 10 | $0x165$ | $x^8 + x^6 + x^5 + x^2 + 1$ |
| 11 | $0x169$ | $x^8 + x^6 + x^5 + x^3 + 1$ |
| 12 | $0x171$ | $x^8 + x^6 + x^5 + x^4 + 1$ |
| 13 | $0x177$ | $x^8 + x^6 + x^5 + x^4 + x^2 + x^1 + 1$ |
| 14 | $0x17b$ | $x^8 + x^6 + x^5 + x^4 + x^3 + x^1 + 1$ |
| 15 | $0x187$ | $x^8 + x^7 + x^2 + x^1 + 1$ |
| 16 | $0x18b$ | $x^8 + x^7 + x^3 + x^1 + 1$ |
| 17 | $0x18d$ | $x^8 + x^7 + x^3 + x^2 + 1$ |
| 18 | $0x19f$ | $x^8 + x^7 + x^4 + x^3 + x^2 + x^1 + 1$ |
| 19 | $0x1a3$ | $x^8 + x^7 + x^5 + x^1 + 1$ |
| 20 | $0x1a9$ | $x^8 + x^7 + x^5 + x^3 + 1$ |
| 21 | $0x1b1$ | $x^8 + x^7 + x^5 + x^4 + 1$ |
| 22 | $0x1bd$ | $x^8 + x^7 + x^5 + x^4 + x^3 + x^2 + 1$ |
| 23 | $0x1c3$ | $x^8 + x^7 + x^6 + x^1 + 1$ |
| 24 | $0x1cf$ | $x^8 + x^7 + x^6 + x^3 + x^2 + x^1 + 1$ |
| 25 | $0x1d7$ | $x^8 + x^7 + x^6 + x^4 + x^2 + x^1 + 1$ |
| 26 | $0x1dd$ | $x^8 + x^7 + x^6 + x^4 + x^3 + x^2 + 1$ |
| 27 | $0x1e7$ | $x^8 + x^7 + x^6 + x^5 + x^2 + x^1 + 1$ |
| 28 | $0x1f3$ | $x^8 + x^7 + x^6 + x^5 + x^4 + x^1 + 1$ |
| 29 | $0x1f5$ | $x^8 + x^7 + x^6 + x^5 + x^4 + x^2 + 1$ |
| 30 | $0x1f9$ | $x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + 1$ |

$$g(x) = g_7 x^7 + g_6 x^6 + g_5 x^5 + g_4 x^4 + g_3 x^3 + g_2 x^2 + g_1 x^1 + g_0.$$

We need to convert an element of $g \in GF(2^8)$, into new basis form. $g$ can be expressed by $\gamma_1 y^{16} + \gamma_0 y \in GF(2^8)/GF(2^4)$, where $\gamma_1$ and $\gamma_0 \in GF(2^4)/GF(2^2)$. Each $\gamma$ can be expressed by $\Gamma_1 z^4 + \Gamma_0 z$, where each $\Gamma \in GF(2^2)/GF(2)$. $\Gamma_1$ and $\Gamma_0$ are considered as a pair of bits $\beta_1$ and $\beta_0$, $\Gamma = \beta_1 w^2 + \beta_0 w$, where $\beta_0$ and $\beta_1 \in \{0, 1\}$. These conversions are possible thanks to field isomorphism between $GF(2^8) \rightarrow GF((2^4)^2) \rightarrow GF(((2^2)^2)^2)$.

The purpose is to find equivalence of $g \in GF(2^8)$ in $GF(((2^2)^2)^2)$.

$$\begin{aligned} g &= \gamma_1 y^{16} + \gamma_0 y \\ &= (\Gamma_3 z^4 + \Gamma_2 z) y^{16} + (\Gamma_1 z^4 + \Gamma_0 z) y \\ &= ((\beta_7 w^2 + \beta_6 w) z^4 + (\beta_5 w^2 + \beta_4 w) z) y^{16} \\ &\quad + ((\beta_3 w^2 + \beta_2 w) z^4 + (\beta_1 w^2 + \beta_0 w) z) y \end{aligned}$$

After putting the g(x) into the equation,

$$g(x) = g_7 x^7 + g_6 x^6 + g_5 x^5 + g_4 x^4$$

$$
\begin{aligned}
&\quad + g_3x^3 + g_2x^2 + g_1x + g_0 \\
&= (g_7x^7 + g_6x^6 + g_5x^5 + g_4x^4)y^{16} \\
&\quad + (g_3x^3 + g_2x^2 + g_1x + g_0)y \\
&= ((g_7x^7 + g_6x^6)z^4 + (g_5x^5 + g_4x^4)z)y^{16} \\
&\quad + ((g_3x^3 + g_2x^2)z^4 + (g_1x + g_0)z)y \\
&= ((g_7x^7w^2 + g_6x^6w)z^4 \\
&\quad + (g_5x^5w^2 + g_4x^4w)z)y^{16} \\
&\quad + ((g_3x^3w^2 + g_2x^2w)z^4 \\
&\quad + (g_1xw^2 + g_0w)z)y
\end{aligned}
$$

We will map $g(x) \rightarrow b(x)$, where $g \in GF(2^8)$ and $b \in GF(((2^2)^2)^2)$.

$$
\begin{aligned}
b(x) &= ((b_7w^2 + b_6w)z^4 + (b_5w^2 + b_4w)z)y^{16} \\
&\quad + ((b_3w^2 + b_2w)z^4 + (b_1w^2 + b_0w)z)y \\
&= (b_7w^2z^4 + b_6wz^4 + b_5w^2z + b_4wz)y^{16} \\
&\quad + ((b_3w^2z^4 + b_2wz^4 + b_1w^2z + b_0wz)y \\
&= b_7\boldsymbol{w^2z^4y^{16}} + b_6\boldsymbol{wz^4y^{16}} + b_5\boldsymbol{w^2zy^{16}} + b_4\boldsymbol{wzy^{16}} \\
&\quad + b_3\boldsymbol{w^2z^4y} + b_2\boldsymbol{wz^4y} + b_1\boldsymbol{w^2zy} + b_0\boldsymbol{wzy}
\end{aligned}
$$

The constant $w^i z^j y^k$ values will be the columns of the mapping matrix $X$, where $i,j,k \in \mathbb{N}$.

$$
X = \begin{pmatrix}
\chi_{0,0} & \chi_{1,0} & \chi_{2,0} & \chi_{3,0} & \chi_{4,0} & \chi_{5,0} & \chi_{6,0} & \chi_{7,0} \\
\chi_{0,1} & \chi_{1,1} & \chi_{2,1} & \chi_{3,1} & \chi_{4,1} & \chi_{5,1} & \chi_{6,1} & \chi_{7,1} \\
\chi_{0,2} & \chi_{1,2} & \chi_{2,2} & \chi_{3,2} & \chi_{4,2} & \chi_{5,2} & \chi_{6,2} & \chi_{7,2} \\
\chi_{0,3} & \chi_{1,3} & \chi_{2,3} & \chi_{3,3} & \chi_{4,3} & \chi_{5,3} & \chi_{6,3} & \chi_{7,3} \\
\chi_{0,4} & \chi_{1,4} & \chi_{2,4} & \chi_{3,4} & \chi_{4,4} & \chi_{5,4} & \chi_{6,4} & \chi_{7,4} \\
\chi_{0,5} & \chi_{1,5} & \chi_{2,5} & \chi_{3,5} & \chi_{4,5} & \chi_{5,5} & \chi_{6,5} & \chi_{7,5} \\
\chi_{0,6} & \chi_{1,6} & \chi_{2,6} & \chi_{3,6} & \chi_{4,6} & \chi_{5,6} & \chi_{6,6} & \chi_{7,6} \\
\chi_{0,7} & \chi_{1,7} & \chi_{2,7} & \chi_{3,7} & \chi_{4,7} & \chi_{5,7} & \chi_{6,7} & \chi_{7,7}
\end{pmatrix}
$$

$$
= \begin{pmatrix}
\chi_0 & \chi_1 & \cdot & \cdot & \cdot & \chi_7
\end{pmatrix}
$$

where each $\chi_i$'s are defined as follow:

$$
\begin{aligned}
\chi_0 &= w^2z^4y^{16}, & \chi_4 &= w^2z^4y^1 \\
\chi_1 &= w^1z^4y^{16}, & \chi_5 &= w^1z^4y^1 \\
\chi_2 &= w^2z^1y^{16}, & \chi_6 &= w^2z^1y^1 \\
\chi_3 &= w^1z^1y^{16}, & \chi_7 &= w^1z^1y^1
\end{aligned}
$$

The matrix $X$ converts an element $g \in GF(2^8)$ into new basis form, which is in the field $GF(((2^2)^2)^2)$. As we can see, the mapping is decided for a choice of the basis denoted as bytes of $(y, z, w)$ values. $X$ denotes the isomorphism bit matrix throughout the chapter.

#### *4.1.1. Generating Affine-based S-Box Table*

S-Box table is the byte array whose length is 256. Each index of the byte array has its own corresponding S-Box value. In order to perform S-Box value of a given index $x$, we need to calculate $Mx^{-1} \oplus c$, where $c$ is the constant byte and $M$ is the circulant bit matrix.

#### *4.1.2. Generating Isomorphism Bit Matrices of Given S-Box*

Composite field is used to compute the multiplicative inverse efficiently. An element in the field $GF(2^8)$ is mapped into an element in the $GF(((2^2)^2)^2)$. The multiplicative inverse is taken in that field. Then the result is converted back to the initial field, which is $GF(2^8)$.

The conversion map $X$ converts an element of the field g $\in GF(2^8)$ to the element of the field b $\in GF(((2^2)^2)^2)$. The inverse of the input is calculated in the domain of the field $GF(((2^2)^2)^2)$. The S-Box formula is $S(x) = M * x^{-1} \oplus c$, where $M$ is the circulant matrix and $c$ is the constant byte. We first change the form of the given input then calculate its inverse. This operation is exactly the same with multiplying calculated $X^{-1}$, where $X^{-1}$ is the inverse of $X$ over $GF(2)$. Then, we multiply by $M \times X$ to change the basis back again. Having $X^{-1}$ and $M \times X$, it is enough to construct an S-Box function.

Instead of finding isomorphism functions between the fields one by one, we searched for basis values ($y$, $z$ and, $w$), which will build $X$ bit matrix at the end. Even though we used a brute-force approach for finding isomorphism bit matrices, we quickly generated all the possible isomorphism bit matrices.

The search space is vast enough, so optimizing all the $8 \times 8$ S-Boxes was impossible. We have chosen one of the irreducible primitive polynomials from the Table 4. After choosing the polynomial, we chose one 8×8 binary circulant matrix and a constant byte value. We generated the S-Box with chosen parameters. Then we found isomorphism bit matrices of generated S-Box. There are more than one isomorphism bit matrix for each generated S-Box. Canright proposed 432 different isomorphism bit matrices for AES S-Box. We used *Python* programming language to generate all the S-Boxes and their corresponding isomorphism bit matrices.

The time-consuming part was to synthesize matrices on hardware. On the hardware side, we have used VHDL language on Xilinx Vivado. Our purpose was to try all the possible isomorphism bit matrices and choose the smallest S-Box in terms of gate-area with good algebraic properties. We have used Xilinx Vivado 21.1 and Xilinx ISE Suite 14.3 tools to synthesize S-Boxes. Synthesizing each isomorphism bit matrix takes approximately one minute in our computer, the specifications of our setup are as follows:

**Processor:** Intel(R) Xeon(R) Gold 62226R CPU 2.90 GHz

**Installed Memory(RAM):** 128 GB
**System Type:** 64-bit Operating System, Windows 10

There are $256 \times 256$ different S-Boxes for only one irreducible polynomial. Moreover, each S-Box has many different isomorphism bit matrices. In case of AES, 432 isomorphism bit matrices are proposed. There are 30 different $8^{th}$ degree irreducible primitive polynomials by Gauss's formula. Therefore the time required to find the smallest affine-based S-Box for the hardware platform (assuming each S-Box has 432 different isomorphism bit matrices) is approximately

$$30 \times 256 \times 256 \times 432 \times 1 \quad min \approx \ 1615,95 \quad years.$$

It requires a considerable amount of computational power to search all the space. Due to a license issue, we were unable to search our space across many cores. One license provides only one synthesis at a time. Therefore, we could only search some of the space but found what we sought. We came up with an S-Box that requires less gate-are and has the same algebraic properties as AES S-Box. Although the composite field approach finds more efficient way to implement an S-Box, some conversion matrices use more hardware gates than naive look-up table implementation. Therefore, we have to be careful while choosing the hardware-oriented bit matrices. The pseudo-algorithm of the finding the most compact $8 \times 8$ S-Box is given in Algorithm 1.

## 4.2. Construction of the Proposed S-Box

According to our search method, we have generated many S-Boxes. Some of them were weak in terms of algebraic properties. Some of them were costly. We focused on the S-Boxes, which have good algebraic properties and are efficiently implementable, among the generated S-Boxes. We desired to select the smallest and the most secure S-Box. We found many equivalent S-Boxes according to the gate-area and security concerns. One of them was chosen to be explained in detail.

**The parameters of the proposed S-Box as follows:**

The irreducible primitive polynomial is

$$g(x) = x^8 + x^7 + x^6 + x^5 + x^4 + x^2 + 1 \in GF(2^8) \quad (2)$$

or $0x1f5$ in hexadecimal notation.

The constant value is $c = 0x09$. The circulant matrix $M$ which is $0x45$ in hexadecimal notation is

$$M = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

The proposed S-Box is constructed in two steps. Let $a$ be the input and $s$ be the output of the S-Box function.

***Step 1:*** Find the multiplicative inverse of $a$ in $GF(2^8)$

$$b = a^{-1}, \quad (3)$$

(if a = 0, then b = 0).

***Step 2:*** Calculate the following affine transformation.

$$\begin{pmatrix} s_7 \\ s_6 \\ s_5 \\ s_4 \\ s_3 \\ s_2 \\ s_1 \\ s_0 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \times \begin{pmatrix} b_7 \\ b_6 \\ b_5 \\ b_4 \\ b_3 \\ b_2 \\ b_1 \\ b_0 \end{pmatrix} \oplus \begin{pmatrix} c_7 \\ c_6 \\ c_5 \\ c_4 \\ c_3 \\ c_2 \\ c_1 \\ c_0 \end{pmatrix}$$

Multiplication of $M$ and $b$ can be also expressed as:

$$b'_i = b_{(i+1)mod8} \oplus b_{(i+3)mod8} \oplus b_{(i+7)mod8} \oplus c_i \quad (4)$$

$$s = s_7 s_6 s_5 s_4 s_3 s_2 s_1 s_0$$
$$c = c_7 c_6 c_5 c_4 c_3 c_2 c_1 c_0$$
$$b = b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0$$

$s_i$'s, $c_i$'s and $b_i$'s are the bits of $s$, $c$, $b$ correspondingly and $0 \leq i < 7$.

All the calculations are computed in $GF(2^8)$. Multiplication and addition of binary matrices do not increase the implementation cost. The difficulty of constructing an S-Box on hardware platforms comes from taking the inverse of an element in the Galois field. Therefore, we focused on finding the multiplicative inverse of an element in the Galois field. The complete $8 \times 8$ proposed S-Box is given in Table 5, and its inverse is given in Table 6.

The input value's initial four bits are represented in the table's first column, and its final four bits are shown in the table's first row. The output of the S-Box function is determined by where the row and column intersect. For example, if input value is $0x7a$, then the intersection of the column with index $'7'$ and the row with index $'a'$ in Table 5 is $0xe5$.

## 4.3. Group Isomorphism of the Proposed S-Box

According to chosen irreducible primitive polynomial $0x1f5$, we have generated many conversion matrices. One of them is chosen for explanation. The parameters of the chosen conversion matrix is $(y, z, w) = (0x13, 0x7a, 0x5d)$. According to parameters, the isomorphism bit matrix of the proposed S-Box is constructed as follow:

$$\chi_0 = 0xf4, \qquad \chi_4 = 0xd2$$
$$\chi_1 = 0xec, \qquad \chi_5 = 0xc7$$
$$\chi_2 = 0x54, \qquad \chi_6 = 0x2e$$
$$\chi_3 = 0xa2, \qquad \chi_7 = 0xd4$$

**Algorithm 1:** Algorithm of Searching Compact S-Box

**Inputs:** $polyList$ ; /* $8^{th}$ degree irr-primitive poly list */
**Output:** $X, M, c, poly$ ; /* X is the isomorphism bit matrix. */
; /* M is the circulant bit matrix. */
; /* c is the constant byte. */

```
aesPr ← getAlgebraicProperties(aesSbox)
minGateArea ← synthesizeSBox(canrightMatrices)
for poly ∈ polyList do
    for M= 0, 1, 2, … , 255 do
        for c= 0, 1, 2, … , 255 do
            ourSbox ← generateSbox(poly,M,c)
            isoList ← generateIsomorphismMatrices(ourSbox,poly,M,c)
            for Y ∈ isoList do
                ourSboxPr ← getAlgebraicProperties(ourSbox)
                gateArea ← synthesizeSBox(Y)
                ; /* compareSboxProperties returns true if ourSboxPr is good as aesPr. */
                if compareSboxProperties(aesPr,ourSboxPr) and gateArea ≤ minGateArea then
                    minGateArea ← gateArea
                    X ← Y
                end
            end
        end
    end
end
return X,M,c,poly
```

**Table 5**
Our proposed S-Box table, generated by the following parameters: Irreducible primitive polynomial is $0x1f5$, the constant is $0x09$ and the circulant matrix is $0x45$.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 09 | ab | fc | a5 | e2 | 3e | 50 | 6c | 6e | 4f | 67 | d3 | bb | 45 | 73 | 43 |
| 1 | c7 | 15 | 35 | 51 | d5 | 7e | bc | ee | dc | 8b | 91 | 1d | fd | 0a | 2d | 34 |
| 2 | 94 | b0 | 31 | e4 | c1 | af | b9 | 0b | 00 | f9 | e7 | f5 | 62 | 11 | 76 | b6 |
| 3 | 12 | 1c | 0c | 71 | 88 | f6 | 21 | 20 | e0 | b3 | bf | 26 | f1 | 7f | c3 | ac |
| 4 | 82 | cf | 7a | 9f | c9 | 3a | d2 | 3d | 28 | cb | f4 | b5 | 68 | 53 | bd | 37 |
| 5 | 1b | 47 | e8 | 80 | 64 | a6 | f0 | ce | df | a8 | 39 | 61 | f7 | 55 | c6 | a3 |
| 6 | 8f | 2c | 23 | b7 | 03 | 40 | 49 | 99 | ba | 9a | 46 | 4d | e9 | b8 | eb | cd |
| 7 | da | 4c | cc | ff | d4 | 13 | 57 | 19 | f8 | 22 | e5 | a9 | 9c | a1 | 42 | c0 |
| 8 | 1e | 78 | 84 | 33 | ef | 93 | 24 | 56 | 38 | c2 | 6f | 3b | be | 36 | d1 | 06 |
| 9 | fb | 3f | 8c | 1f | f2 | 52 | 70 | d8 | 7b | 79 | 0d | b4 | 60 | 18 | 75 | 8a |
| a | 9d | 66 | 25 | a0 | ca | dd | aa | 87 | 63 | 16 | e6 | 85 | fa | 5e | 86 | 17 |
| b | a4 | e1 | 4a | 5a | d9 | 90 | 69 | d7 | 44 | fe | b1 | 14 | 96 | 8e | ec | 10 |
| c | 04 | 01 | f3 | 72 | 5d | ed | c4 | 59 | ad | 41 | 9b | 0e | 89 | 6b | 98 | 2b |
| d | de | b2 | 2e | 95 | 27 | 5c | 81 | 65 | c8 | 4b | 6a | 1a | 7c | 4e | 30 | 0f |
| e | ae | c5 | 83 | 6d | 32 | db | 54 | 9e | 02 | 08 | 8d | a7 | 05 | d6 | 29 | 77 |
| f | ea | a2 | 5f | 7d | d0 | 97 | 48 | 5b | 92 | e3 | 58 | 07 | 2f | 74 | 2a | 3c |

And the conversion map is

$$X = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

This map $X$ converts an element in the field $GF(((2^2)^2)^2)$ into an element of $GF(2^8)$.

While looking for S-Box function of a given input, first we need to multiply by $X^{-1}$ to convert into the basis for $GF(((2^2)^2)^2)$ and calculate its inverse in $GF(2)$. After that, we need to change the basis back again and perform the affine transformation. Therefore, we multiply by $M \times X$ directly.

**Table 6**
Our proposed inverse S-Box table, generated by the following parameters: Irreducible primitive polynomial is $0x1f5$, the constant is $0x09$ and the circulant matrix is $0x45$.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 28 | c1 | e8 | 64 | c0 | ec | 8f | fb | e9 | 00 | 1d | 27 | 32 | 9a | cb | df |
| 1 | bf | 2d | 30 | 75 | bb | 11 | a9 | af | 9d | 77 | db | 50 | 31 | 1b | 80 | 93 |
| 2 | 37 | 36 | 79 | 62 | 86 | a2 | 3b | d4 | 48 | ee | fe | cf | 61 | 1e | d2 | fc |
| 3 | de | 22 | e4 | 83 | 1f | 12 | 8d | 4f | 88 | 5a | 45 | 8b | ff | 47 | 05 | 91 |
| 4 | 65 | c9 | 7e | 0f | b8 | 0d | 6a | 51 | f6 | 66 | b2 | d9 | 71 | 6b | dd | 09 |
| 5 | 06 | 13 | 95 | 4d | e6 | 5d | 87 | 76 | fa | c7 | b3 | f7 | d5 | c4 | ad | f2 |
| 6 | 9c | 5b | 2c | a8 | 54 | d7 | a1 | 0a | 4c | b6 | da | cd | 07 | e3 | 08 | 8a |
| 7 | 96 | 33 | c3 | 0e | fd | 9e | 2e | ef | 81 | 99 | 42 | 98 | dc | f3 | 15 | 3d |
| 8 | 53 | d6 | 40 | e2 | 82 | ab | ae | a7 | 34 | cc | 9f | 19 | 92 | ea | bd | 60 |
| 9 | b5 | 1a | f8 | 85 | 20 | d3 | bc | f5 | ce | 67 | 69 | ca | 7c | a0 | e7 | 43 |
| a | a3 | 7d | f1 | 5f | b0 | 03 | 55 | eb | 59 | 7b | a6 | 01 | 3f | c8 | e0 | 25 |
| b | 21 | ba | d1 | 39 | 9b | 4b | 2f | 63 | 6d | 26 | 68 | 0c | 16 | 4e | 8c | 3a |
| c | 7f | 24 | 89 | 3e | c6 | e1 | 5e | 10 | d8 | 44 | a4 | 49 | 72 | 6f | 57 | 41 |
| d | f4 | 8e | 46 | 0b | 74 | 14 | ed | b7 | 97 | b4 | 70 | e5 | 18 | a5 | d0 | 58 |
| e | 38 | b1 | 04 | f9 | 23 | 7a | aa | 2a | 52 | 6c | f0 | 6e | be | c5 | 17 | 84 |
| f | 56 | 3c | 94 | c2 | 4a | 2b | 35 | 5c | 78 | 29 | ac | 90 | 02 | 1c | b9 | 73 |

$$M \times X = \begin{pmatrix} 0&1&0&0&0&1&0&1 \\ 1&0&0&0&1&0&1&0 \\ 0&0&0&1&0&1&0&1 \\ 0&0&1&0&1&0&1&0 \\ 0&1&0&1&0&1&0&0 \\ 1&0&1&0&1&0&0&0 \\ 0&1&0&1&0&0&0&1 \\ 1&0&1&0&0&0&1&0 \end{pmatrix} \times \begin{pmatrix} 1&1&0&1&1&1&0&1 \\ 1&1&1&0&1&1&0&1 \\ 1&1&0&1&0&0&1&0 \\ 1&0&1&0&1&0&0&1 \\ 0&1&0&0&0&0&1&0 \\ 1&1&1&0&0&1&1&1 \\ 0&0&0&1&1&1&1&0 \\ 0&0&0&0&0&1&0&0 \end{pmatrix} = \begin{pmatrix} 0&0&0&0&1&1&1&0 \\ 1&0&0&0&0&0&0&1 \\ 0&1&0&0&1&0&1&0 \\ 1&0&0&0&1&1&1&0 \\ 1&0&1&0&0&0&1&1 \\ 0&1&0&0&1&1&0&1 \\ 0&1&0&0&0&0&0&0 \\ 0&0&0&1&0&0&0&1 \end{pmatrix}$$

The inverse of $X$ in $GF(2)$ computes its reverse mapping. The inverse map is

$$X^{-1} = \begin{pmatrix} 0&1&1&0&1&1&1&1 \\ 0&1&0&1&0&0&0&1 \\ 1&0&0&0&0&1&0&1 \\ 0&1&0&0&0&1&1&1 \\ 0&0&0&1&1&1&0&1 \\ 0&0&0&0&0&0&0&1 \\ 0&1&0&1&1&0&0&1 \\ 1&1&1&0&0&1&0&1 \end{pmatrix}.$$

## 4.4. Results & Comparisons

We compared the results of our study to those of other researchers who had already worked on this similar topic, using the same setup and platforms. We downloaded the Xilinx ISE Design Suite 14.4 for Virex-4, Virex-5, and Virex-6 FPGAs and the Vivado 2021.1 for Artix-7 and Virtex-7 FPGAs. During synthesis, we used the identical FPGA components in the same configuration. Since implementing only S-Box occupies very small amount of space on an FPGA, the synthesis frequencies of the designs do not make sense. Therefore, we did not discuss synthesis frequency or synthesis period in our research.

The naive look-up table design is the complete $8 \times 8$ S-Box table, a byte array of size 256. Canright's matrices design implements AES S-Box in composite field arithmetic. The isomorphism bit matrices are taken from [10].

### *4.4.1. Naive Look-up Table Implementation*

This approach makes use of a full-byte array of size 256. This byte array can be distributed to LUTs or mapped into a BRAM. Some FPGAs only map to BRAM, which is both inefficient and undesirable. Since one BRAM, which is 18-Kbit or 32-Kbit, will be used for only the 256-bytes S-Box. As a result, it is preferable to design S-Boxes in a distributed manner. Therefore, the distributed cost of the S-Box is taken into account in our comparison tables.

### *4.4.2. Canright's Matrices for AES S-Box*

432 different isomorphism bit matrices are proposed by [10] for AES algorithm. He proposed the smallest matrix in terms of gate-area in early 2005. The AES S-Box's irreducible primitive polynomial is $0x11b$, and the proposed conversion parameters are $(y, z, w) = (0xff, 0x5c, 0xbd)$. The most compact isomorphism bit matrix of the AES S-Box

is proposed by [10] as follow:

$$\chi_0 = 0x64,$$
$$\chi_1 = 0x78,$$
$$\chi_2 = 0x6e,$$
$$\chi_3 = 0x8c,$$
$$\chi_4 = 0x68,$$
$$\chi_5 = 0x29,$$
$$\chi_6 = 0xde,$$
$$\chi_7 = 0x60$$

$$X = \chi_0\chi_1\chi_2\chi_3\chi_4\chi_5\chi_6\chi_7$$

The map $X$ converts an element from the field $GF(((2^2)^2)^2)$ into an element of $GF(2^8)$. The inverse of $X$ in $GF(2)$ computes its reverse mapping. $M$ is the circulant matrix of AES S-Box. The synthesis results of the given $X^{-1}$ and $M \times X$ matrices are used for comparison tables.

The conversion and its inverse map for AES [10] are

$$X = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

$$X^{-1} \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

$$M \times X = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

The synthesis result of the proposed S-Box and previous AES S-Box implementation results are compared in the Table 7 for the platform Virtex-4. The composite field approach of implementing the proposed S-Box is executed on Xilinx ISE Design Suite 14.4. Virtex-4 FPGA LUTs, as explained in the previous section, consist of 4-independent inputs and returns 1-bit output.The complete circuit is optimized by the Xilinx synthesis tool at the end. We observed that the isomorphism bit matrices of the proposed S-Box are optimized more than AES S-Box in terms of the number of used LUTs.

The AES S-Box implementation on Virtex-4 FPGAs was optimized by the researchers using a variety of techniques. Table 7 shows that for Virtex-4 xc4vlx25-10ff668, our synthesis results are 21% more compact than [24]. For Virtex-4 xc4vlx200-11ff1513, our circuit costs 24% less than Priya's results. [29] used mux-based S-Box implementation in their design. The multiplication in $GF((2^2)^2)$ is computed with multiplexers instead of XOR gates. The study reduced the number of multipliers and decreased the critical path [29]. [24] proposed a design which computes the multiplicative inverse in composite fields to some point. The multiplications in $GF((2^2)^2)$ is stored in pre-calculated LUTs to decrease the number of LUTs in total design. According to [10]'s matrices, our S-Box is 11.76% and 5.9% smaller respectively on Virtex-4 xc4vlx25-10ff668 and Virtex xc4vlx200-11ff1513 FPGAs.

Virtex-5 and Virtex-6 FPGAs use the more recent iteration of LUTs. The Virtex-5 and Virtex-6 LUTs can use 6-independent inputs. With this improvement, FPGAs can produce more compact results; as seen in Table 8, the number of LUTs significantly dropped according to Virtex-4 FPGAs. The proposed S-Box implementation results are identical to those of other researchers. The only design that costs more than 32 LUT is [28] design. Multiplication block is implemented with five-stage pipelining in the design of [28]. Others researchers in Table 8 used the composite field approach and found the same results.

The proposed S-Box results differ from those of the other researchers for the new FPGA models Artix-7 and Virtex-7. This difference comes from the synthesis tool's ability to optimize the circuit. Compared to Pradeep's and Huy's designs for Artix-7 and Virtex-7, our result is 21% and 96% better, respectively. [27] preferred to use half-way composite

**Table 7**
Synthesis result comparison of the proposed S-Box and the best optimized AES S-Box designs on Virtex-4 FPGAs for Only One Byte of Input

| Year | Used Resource | | Design | Platform |
|---|---|---|---|---|
| 2020 | 35 Slices | 69 LUTs | [24] | Virtex4 - xc4vlx25-10ff668 (Xilinx ISE Design Suite 14.4 System Edition) |
| | 64 Slices | 128 LUTs | Naive Look-up Table 4.4.1 | |
| | 34 Slices | 60 LUTs | Canright's Matrices 4.4.2 | |
| 2023 | 30 Slices | 57 LUTs | Our work, Table 5 | |
| 2017 | 37 Slices | 71 LUTs | [29] | Virtex4 - xc4vlx200-11ff1513 (Xilinx ISE Design Suite 14.4 System Edition) |
| | 64 Slices | 128 LUTs | Naive Look-up Table 4.4.1 | |
| | 34 Slices | 60 LUTs | Canright's Matrices 4.4.2 | |
| 2023 | 32 Slices | 57 LUTs | Our work, Table 5 | |

**Table 8**
Synthesis result comparison of the proposed S-Box and the best optimized AES S-Box designs on Virtex-5 and Virtex-6 FPGAs for Only One Byte of Input

| Year | Used Resource | Design | Platform |
|---|---|---|---|
| 2008 | 32 LUTs | [9] | Virtex5 - xc5vlx50t-3ff36 (Xilinx ISE Design Suite 14.4 System Edition) |
| 2015 | 32 LUTs | [25] | |
| 2019 | 172 LUTs | [28] | |
| | 32 LUTs | Naive Look-up Table 4.4.1 | |
| | 32 LUTs | Canright's Matrices 4.4.2 | |
| 2023 | 32 LUTs | Our work, Table 5 | |
| 2015 | 32 LUTs | [37] | Virtex6 - xc6vlx240t-3ff784 (Xilinx ISE Design Suite 14.4 System Edition) |
| | 32 LUTs | Naive Look-up Table 4.4.1 | |
| | 32 LUTs | Canright's Matrices 4.4.2 | |
| 2023 | 32 LUTs | Our work, Table 5 | |

field. The inverse operation is taken in the field $GF((4^2)^2)$ instead of $GF(((2^2)^2)^2)$. Canright's matrices cost at least same or more than the suggested S-Box on all platforms. The number of used LUTs for Virtex-7 and Artix-7 are given in Table 9 in detail.

These tables only show the required number of LUTs for a single byte. For the latest FPGA models, each byte requires around 32 LUTs. AES uses blocks of a 16-byte size. In order to implement the AES substitution layer, hardware platforms need 16 pipelined S-Box tables for round function and 4 pipelined S-Box tables for key schedule, which means that values in the table will be multiplied by 20. The AES core is also used more than once for high-throughput devices. Some systems use even 256 copies of AES core. In this case, the values in the table will be multiplied by 5120. Therefore, even minor LUT enhancement will significantly impact high-throughput systems.

The implementation of [17] uses total 2444 LUTs for AES core. He used naive-way implementation of S-Box in his design. If they use our S-Box instead of Rijndael S-Box, he would save 180 LUTs more. Since implementing one S-Box in a naive-way costs 40 LUTs on Virtex-7 FPGAs. Our S-Box costs only 31 LUTs. Each round use 20 S-Box tables, therefore they would save $20 \times 9 = 180$ LUTs.

## 5. Security of the Proposed S-Box

The S-Box layer has a significant impact on security of the algorithm. Since this layer is the only non-linear layer of the algorithm. The S-Box provides security against linear and differential cryptanalysis. In addition to the hardware cost, the security of the proposed S-Box is also taken into account.

**Definition 5.1 (Difference Distribution Table, [6]).** *Difference Distribution table of* $8 \times 8$ *S-Box S is defined as follow: DDT is an* $2^8 \times 2^8$ *matrix with row* $i \in \mathbb{F}_2^8$ *and column* $j \in \mathbb{F}_2^8$ *equal to*

$$\#\{x \in \{0,1\}^8 \mid S(x) \oplus S(x \oplus i) = j\}, \tag{5}$$

*for any pair (i, j). The highest value in the difference distribution table, without considering the first entry, is called* ***differential uniformity of the S-Box****.*

Having small differential uniformity makes the S-Box table strong against differential cryptanalysis. The differential cryptanalysis is proposed by Biham and Shamir in 1990. For more information about difference distribution table and differential attack, see [6].

**Definition 5.2 (Linear Approximation Table, [23]).** *Linear approximation table of* $8 \times 8$ *S-Box S is defined as follow:*

*LAT is a* $2^8 \times 2^8$ *matrix with row* $i \in \mathbb{F}_2^8$ *and column* $j \in \mathbb{F}_2^8$ *equal to*

$$\left[\sum_{x=0}^{255} (i \cdot x) \oplus \left(j \cdot S(x)\right)\right] - 128. \tag{6}$$

**Table 9**
Synthesis result comparison of the proposed S-Box and the best optimized AES S-Box designs on Virtex-7 and Artix-7 FPGAs for Only One Byte of Input

| Year | Used Resource | Design | Platform |
|---|---|---|---|
| 2019 | 39 LUTs | [27] | Artix7 - xc7a200tfbg676-2 (Xilinx Vivado 2021.1 ML Edition) |
| | 40 LUTs | Naive Look-up Table 4.4.1 | |
| | 32 LUTs | Canright's Matrices 4.4.2 | |
| 2023 | 31 LUTs | Our work, Table 5 | |
| 2019 | 61 LUTs | [18] | Virtex7 - xc7vlx980t-2ffg1926 (Xilinx Vivado 2021.1 ML Edition) |
| | 40 LUTs | Naive Look-up Table 4.4.1 | |
| | 32 LUTs | Canright's Matrices 4.4.2 | |
| 2023 | 31 LUTs | Our work, Table 5 | |

*The maximum absolute value of linear approximation table, except the first entry, is called* ***linear uniformity of the S-Box****.*

The S-Boxes whose linear uniformity is small have strong resistance against linear cryptanalysis. The linear cryptanalysis is discovered by Matsui in 1993. For more information about linear approximation table and linear cryptanalysis, see [23].

**Definition 5.3 (Boomerang Connectivity Table, [11]).** *Boomerang connectivity table of* $8 \times 8$ *S-Box S is defined as follow:*

*BCT is a* $2^8 \times 2^8$ *matrix with row* $i \in \mathbb{F}_2^8$ *and column* $j \in \mathbb{F}_2^8$ *equal to*

$$\left| \left\{ x \in \mathbb{F}_2^8 \mid S^{-1}\big(S(x) \oplus j\big) \oplus S'\big(S(x \oplus i) \oplus j\big) = i \right\} \right|. \quad (7)$$

The highest value in the boomerang connectivity table, ignoring the elements of the first row and column, is called **boomerang uniformity of the S-Box**. Lower boomerang uniformity provides higher security against the boomerang attacks. For detailed explanation of boomerang attacks and connectivity table, see [42] and [8].

The algebraic properties of the proposed S-Box are analyzed and the details are summarized in the Table 10.

## 6. Conclusion

In this work, we have used the same optimization methods that Canright used to optimize the S-Box layer of AES. We aimed to find a more compact $8 \times 8$ S-Box that provides at least the same security level as the AES S-Box. Since the search space is vast enough, analyzing all the $8 \times 8$ S-Boxes was computationally infeasible. We proposed another S-Box which provides the same level of security against cryptanalysis techniques with an 11.76% less gate-area on Virtex-4 FPGAs. Our S-Box costs 3.125% fewer gates on Artix-7 and Virtex-7 FPGAs and uses the equal resource on Virtex-5 and Virtex-6 FPGAs when it is compared to AES S-Box. The algebraic properties of the proposed S-Box confirm that it provides the same level of security as the AES S-Box. Thus, the proposed S-Box is a better alternative to the AES S-Box as it offers improved efficiency on Virtex-4, Virtex-7, and Artix-7 FPGAs.

## Acknowledgment

The work of Cihangir Tezcan has been supported by TUBITAK 1001 Project under the grant number 121E228 and by Middle East Technical University Scientific Research Projects Coordination Unit under grant number AGEP-704-2023-11294.

## References

[1] Abdelrahman, A.A., Fouad, M.M., Dahshan, H., Mousa, A.M., 2017. High performance cuda aes implementation: A quantitative performance analysis approach, in: 2017 Computing Conference, pp. 1077–1085. doi:10.1109/SAI.2017.8252225.

[2] AMD Xilinx Company, 2022. Vivado design suite 7 series fpga and zynq-7000 soc libraries guide (ug953). https://docs.xilinx.com/r/en-US/ug953-vivado-7series-libraries. URL: https://docs.xilinx.com/r/en-US/ug953-vivado-7series-libraries.

[3] An, S.W., SEO, S., 2020. Highly efficient implementation of block ciphers on graphic processing units for massively large data. Applied Sciences 10, 3711. doi:10.3390/app10113711.

[4] An, S.W., Seo, S.C., 2020. Study on optimizing block ciphers (aes, cham) on graphic processing units, in: 2020 IEEE International Conference on Consumer Electronics - Asia (ICCE-Asia), pp. 1–4. doi:10.1109/ICCE-Asia49877.2020.9276980.

[5] Aziz, A., Ikram, N., 2007. Memory efficient implementation of AES s-boxes on FPGA. J. Circuits Syst. Comput. 16, 603–611. URL: https://doi.org/10.1142/S0218126607003873, doi:10.1142/S0218126607003873.

[6] Biham, E., Shamir, A., 1990. Differential cryptanalysis of des-like cryptosystems, in: Menezes, A., Vanstone, S.A. (Eds.), Advances in Cryptology - CRYPTO '90, 10th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1990, Proceedings, Springer. pp. 2–21. URL: https://doi.org/10.1007/3-540-38424-3_1, doi:10.1007/3-540-38424-3_1.

[7] Bouhraoua, A., 2010. Design feasibility study for a 500 gbits/s advanced encryption standard cipher/decipher engine. Computers and Digital Techniques, IET 4, 334 – 348. doi:10.1049/iet-cdt.2009.0023.

[8] Boura, C., Canteaut, A., 2018. On the boomerang uniformity of cryptographic sboxes. IACR Trans. Symmetric Cryptol. 2018, 290–310. URL: https://doi.org/10.13154/tosc.v2018.i3.290-310, doi:10.13154/tosc.v2018.i3.290-310.

[9] Bulens, P., Standaert, F., Quisquater, J., Pellegrin, P., Rouvroy, G., 2008. Implementation of the AES-128 on virtex-5 fpgas, in: Vaudenay, S. (Ed.), Progress in Cryptology - AFRICACRYPT 2008, First International Conference on Cryptology in Africa, Casablanca, Morocco, June 11-14, 2008. Proceedings, Springer. pp. 16–26. URL: https://doi.org/10.1007/978-3-540-68164-9_2, doi:10.1007/978-3-540-68164-9_2.

[10] Canright, D., 2005. A very compact s-box for AES, in: Rao, J., Sunar, B. (Eds.), Cryptographic Hardware and Embedded Systems - CHES 2005, 7th International Workshop, Edinburgh, UK, August

**Table 10**
The security properties of the proposed S-Box and AES S-Box.

| Algebraic Property | AES S-Box | Our S-Box, Table 5 | AES Inverse S-Box | Our Inverse S-Box, Table 6 |
|---|---|---|---|---|
| Boomerang Uniformity | 6 | 6 | 6 | 6 |
| Differential Branch Number | 2 | 2 | 2 | 2 |
| Differential Uniformity | 4 | 4 | 4 | 4 |
| Has Fixed Points | False | False | False | False |
| Has Linear Structure | False | False | False | False |
| Is Almost Bent | False | False | False | False |
| Is Almost Perfect Nonlinear | False | False | False | False |
| Is Balanced | True | True | True | True |
| Is Bent | False | False | False | False |
| Is Involution | False | False | False | False |
| Is Monomial Function | False | False | False | False |
| Is Permutation | True | True | True | True |
| Linear Branch Number | 2 | 2 | 2 | 2 |
| Linearity | 32 | 32 | 32 | 32 |
| Minimal Degree | 7 | 7 | 7 | 7 |
| Max. Degree | 7 | 7 | 7 | 7 |
| Max. Diff. Prob. | 0.015625 | 0.015625 | 0.015625 | 0.015625 |
| Max. Diff. Prob. Absolute | 4 | 4 | 4 | 4 |
| Max. Linear Bias Absolute | 16 | 16 | 16 | 16 |
| Max. Linear Bias Relative | 0.625 | 0.625 | 0.625 | 0.625 |
| Non-Linearity | 112 | 112 | 112 | 112 |

29 - September 1, 2005, Proceedings, Springer. pp. 441–455. URL: https://doi.org/10.1007/11545262_32, doi:10.1007/11545262_32.

[11] Cid, C., Huang, T., Peyrin, T., Sasaki, Y., Song, L., 2018. Boomerang connectivity table: A new cryptanalysis tool, in: Nielsen, J.B., Rijmen, V. (Eds.), Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part II, Springer. pp. 683–714. URL: https://doi.org/10.1007/978-3-319-78375-8_22, doi:10.1007/978-3-319-78375-8_22.

[12] Criado, J.M.G., Vega-Rodríguez, M.A., Sánchez-Pérez, J.M., Pulido, J.A.G., 2014. Hardware security platform for multicast communications. J. Syst. Archit. 60, 11–21. URL: https://doi.org/10.1016/j.sysarc.2013.11.007, doi:10.1016/j.sysarc.2013.11.007.

[13] Daemen, J., Rijmen, V., 2002. The Design of Rijndael: AES - The Advanced Encryption Standard. Information Security and Cryptography, Springer. URL: https://doi.org/10.1007/978-3-662-04722-4, doi:10.1007/978-3-662-04722-4.

[14] Dhanalakshmi, K.S., Padmavathi, R.A., 2022. A survey on vlsi implementation of aes algorithm with dynamic s-box. Journal of Applied Security Research 17, 241–256. URL: https://doi.org/10.1080/19361610.2020.1870403, doi:10.1080/19361610.2020.1870403, arXiv:https://doi.org/10.1080/19361610.2020.1870403.

[15] Gueron, S., 2010. Intel advanced encryption standard (aes) new instructions set, white paper 323641-001. https://www.intel.com/content/dam/doc/white-paper/advanced-encryption-standard-new-instructions-set-paper.pdf. URL: https://www.intel.com/content/dam/doc/white-paper/advanced-encryption-standard-new-instructions-set-paper.pdf.

[16] Hasija, T., Kaur, A., Ramkumar, K.R., Sharma, S., Mittal, S., Singh, B., 2023. A survey on performance analysis of different architectures of aes algorithm on fpga, in: Agrawal, R., Kishore Singh, C., Goyal, A., Singh, D.K. (Eds.), Modern Electronics Devices and Communication Systems, Springer Nature Singapore, Singapore. pp. 39–54.

[17] Hussain, U., Jamal, H., 2012. An efficient high throughput fpga implementation of aes for multi-gigabit protocols, in: 2012 10th International Conference on Frontiers of Information Technology, pp. 215–218. doi:10.1109/FIT.2012.45.

[18] Huy, D.Q., Duc, N.M., Khai, L.D., Lung, V.D., 2019. Hardware implementation of AES with s-box using composite-field for WLAN systems, in: 2019 IEEE-RIVF International Conference on Computing and Communication Technologies, RIVF 2019, Danang, Vietnam, March 20-22, 2019, IEEE. pp. 1–6. URL: https://doi.org/10.1109/RIVF.2019.8713711, doi:10.1109/RIVF.2019.8713711.

[19] Iyer, N., Anandmohan, P., Poornaiah, D., Kulkarni, V., 2011. Computational Intelligence and Information Technology. volume 250. chapter Efficient hardware architectures for AES on FPGA. pp. 249–257. doi:10.1007/978-3-642-25734-6_37.

[20] Knudsen, L.R., Dobbertin, H., Robshaw, M.J.B., 2004. The cryptanalysis of the AES - A brief survey, in: Dobbertin, H., Rijmen, V., Sowa, A. (Eds.), Advanced Encryption Standard - AES, 4th International Conference, AES 2004, Bonn, Germany, May 10-12, 2004, Revised Selected and Invited Papers, Springer. pp. 1–10. URL: https://doi.org/10.1007/11506447_1, doi:10.1007/11506447_1.

[21] Kundi, D., Aziz, A., Ikram, N., 2016. A high performance st-box based unified AES encryption/decryption architecture on FPGA. Microprocess and Microsystems 41, 37–46. URL: https://doi.org/10.1016/j.micpro.2015.11.015, doi:10.1016/j.micpro.2015.11.015.

[22] Madhavapandian, S., MaruthuPandi, P., 2020. Fpga implementation of highly scalable aes algorithm using modified mix column with gate replacement technique for security application in tcp/ip. Microprocessors and Microsystems 73, 102972. URL: https://www.sciencedirect.com/science/article/pii/S0141933119304703, doi:https://doi.org/10.1016/j.micpro.2019.102972.

[23] Matsui, M., 1993. Linear cryptanalysis method for DES cipher, in: Helleseth, T. (Ed.), Advances in Cryptology - EUROCRYPT '93, Workshop on the Theory and Application of of Cryptographic Techniques, Lofthus, Norway, May 23-27, 1993, Proceedings, Springer. pp. 386–397. URL: https://doi.org/10.1007/3-540-48285-7_33, doi:10.1007/3-540-48285-7_33.

[24] Murugan, A., Karthigaikumar, Priya, S.S., 2020. Fpga implementation of hardware architecture with aes encryptor using sub-pipelined s-box techniques for compact applications, in: Automatika, Taylor and Francis. pp. 682–693. URL: https://doi.org/10.1080/00051144.2020.1816388, doi:10.1080/00051144.2020.1816388.

[25] Nadjia, A., Mohamed, A., 2015. Efficient implementation of aes s-box in lut-6 fpgas, in: 2015 4th International Conference on Electrical Engineering (ICEE), IEEE. pp. 1–4. URL: https://doi.org/10.1109/INTEE.2015.7416679, doi:10.1109/INTEE.2015.7416679.

[26] Nishikawa, N., Amano, H., Iwai, K., 2017. Implementation of bitsliced aes encryption on cuda-enabled gpu, in: Lecture Notes in Computer Science, pp. 273–287. doi:10.1007/978-3-319-64701-2_20.

[27] Pradeep, A., Mohanty, V., Subramaniam, A.M., Rebeiro, C., 2019. Revisiting AES sbox composite field implementations for fpgas. IEEE Embed. Syst. Lett. 11, 85–88. URL: https://doi.org/10.1109/LES.2019.2899113, doi:10.1109/LES.2019.2899113.

[28] Priya, S.S., Karthigaikumar, 2019. Fpga implementation of high speed compact s-box, in: International Journal of Pure and Applied Mathematics, IEEE. pp. 1703–1711.

[29] Priya, S.S., Karthigaikumar, Mangai, S., Das, K.G., 2017. An efficient hardware architecture for high throughput AES encryptor using MUX based sub pipelined s-box. Wirel. Pers. Commun. 94, 2259–2273. URL: https://doi.org/10.1007/s11277-016-3385-7, doi:10.1007/s11277-016-3385-7.

[30] Rais, M., Qasim, S.M., 2010. Virtex-5 fpga implementation of advanced encryption standard algorithm, in: AIP Conference Proceedings, pp. 201–205. doi:10.1063/1.3459750.

[31] Raja, L., Masanan, K., 2020. Enhancing the security of aes through small scale confusion operations for data communication. Microprocessors and Microsystems 75, 103041. URL: https://www.sciencedirect.com/science/article/pii/S0141933119306568, doi:https://doi.org/10.1016/j.micpro.2020.103041.

[32] Razaq, A., Alhamzi, G., Abbas, S., Ahmad, M., Razzaqe, A., 2023. Secure communication through reliable s-box design: A proposed approach using coset graphs and matrix operations. Heliyon 9, e15902. URL: https://www.sciencedirect.com/science/article/pii/S2405844023031092, doi:https://doi.org/10.1016/j.heliyon.2023.e15902.

[33] Razzaque, A., Razaq, A., Farooq, S.M., Masmali, I., Faraz, M.I., 2023. An efficient s-box design scheme for image encryption based on the combination of a coset graph and a matrix transformer. Electronic Research Archive 31, 2708–2732. URL: https://www.aimspress.com/article/doi/10.3934/era.2023137, doi:10.3934/era.2023137.

[34] Reddy, S.K., Sakthivel, R., Praneeth, P., 2011. Vlsi implementation of aes crypto processor for high throughput. International journal of advanced engineering sciences and technologies 6, 022–026.

[35] Saggese, G.P., Mazzeo, A., Mazzocca, N., Strollo, A.G.M., 2003. An fpga-based performance analysis of the unrolling, tiling, and pipelining of the AES algorithm, in: Cheung, P.Y.K., Constantinides, G.A., de Sousa, J.T. (Eds.), Field Programmable Logic and Application, 13th International Conference, FPL 2003, Lisbon, Portugal, September 1-3, 2003, Proceedings, Springer. pp. 292–302. URL: https://doi.org/10.1007/978-3-540-45234-8_29, doi:10.1007/978-3-540-45234-8_29.

[36] Satoh, A., Morioka, S., Takano, K., Munetoh, S., 2001. A compact rijndael hardware architecture with s-box optimization, in: Boyd, C. (Ed.), Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings, Springer. pp. 239–254. URL: https://doi.org/10.1007/3-540-45682-1_15, doi:10.1007/3-540-45682-1_15.

[37] Savalam, C., Korapati, P., 2015. Implementation and design of aes s-box on fpga. International Journal of Research in Engineering and Science 3, 9–14.

[38] Sheikhpour, S., Mahani, A., Bagheri, N., 2021. Reliable advanced encryption standard hardware implementation: 32- bit and 64-bit data-paths. Microprocessors and Microsystems 81, 103740. URL: https://www.sciencedirect.com/science/article/pii/S0141933120308851, doi:https://doi.org/10.1016/j.micpro.2020.103740.

[39] Sunil, C., Jan, M., 2011. Counting irreducible polynomials over finite fields using the inclusion-exclusion principle, in: Mathematics Magazine, arXiv. pp. 369–371. URL: https://arxiv.org/abs/1001.0409, doi:10.48550/ARXIV.1001.0409.

[40] Tezcan, C., 2021. Optimization of advanced encryption standard on graphics processing units. IEEE Access 9, 67315–67326. URL: https://doi.org/10.1109/ACCESS.2021.3077551, doi:10.1109/ACCESS.2021.3077551.

[41] Tezcan, C., 2022. Key lengths revisited: Gpu-based brute force cryptanalysis of des, 3des, and PRESENT. J. Syst. Archit. 124, 102402. URL: https://doi.org/10.1016/j.sysarc.2022.102402, doi:10.1016/j.sysarc.2022.102402.

[42] Wagner, D.A., 1999. The boomerang attack, in: Knudsen, L.R. (Ed.), Fast Software Encryption, 6th International Workshop, FSE '99, Rome, Italy, March 24-26, 1999, Proceedings, Springer. pp. 156–170. URL: https://doi.org/10.1007/3-540-48519-8_12, doi:10.1007/3-540-48519-8_12.

[43] Wang, Y., Ha, Y., 2013. Fpga-based 40.9-gbits/s masked AES with area optimization for storage area network. IEEE Trans. Circuits Syst. II Express Briefs 60-II, 36–40. URL: https://doi.org/10.1109/TCSII.2012.2234891, doi:10.1109/TCSII.2012.2234891.